

6.2 实验二：训练环境搭建与模型训练

6.2.1 实验目的

本实验旨在使学生掌握 YOLOv5 目标检测模型训练的完整流程,包括 Python 虚拟环境的构建与配置、数据集的准备与加载、模型训练的参数设置与优化等关键环节;通过实操完成从数据到模型的转化,最终生成可用于嵌入式部署的权重文件 (best.pt), 为后续模型转换与边缘端推理奠定基础。

6.2.2 实验说明

本实验主要面向于深度学习目标检测模型的训练环节,采用改进版 YOLOv5 进行坦克目标识别任务。实验将通过 Miniconda 构建独立的训练环境,安装所需依赖,配置数据集并执行训练。最终生成权重文件 best.pt, 为后续模型转换与部署做好准备。

6.2.3 实验步骤

步骤一：安装 Miniconda

Miniconda 是一个轻量级的 Python 环境管理工具,用于创建独立的虚拟环境,便于不同项目之间的依赖隔离。

Miniconda 下载地址: <https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/>

Miniconda 下载如图 6.10 所示:

Miniconda3-py38_22.11.1-Linux-s390x.sh	57.9 MiB	2025-02-14 03:49
Miniconda3-py38_22.11.1-Linux-x86_64.sh	61.6 MiB	2025-02-14 03:49
Miniconda3-py38_22.11.1-MacOSX-arm64.pkg	42.9 MiB	2025-02-14 03:49
Miniconda3-py38_22.11.1-MacOSX-arm64.sh	43.2 MiB	2025-02-14 03:49
Miniconda3-py38_22.11.1-MacOSX-x86_64.pkg	44.4 MiB	2025-02-14 03:49
Miniconda3-py38_22.11.1-MacOSX-x86_64.sh	44.6 MiB	2025-02-14 03:49
Miniconda3-py38_22.11.1-Windows-x86_64.exe	52.5 MiB	2025-02-14 03:49
Miniconda3-py38_23.1.0-Linux-aarch64.sh	48.2 MiB	2025-02-14 03:49
Miniconda3-py38_23.1.0-Linux-ppc64le.sh	48.5 MiB	2025-02-14 03:49
Miniconda3-py38_23.1.0-Linux-s390x.sh	61.5 MiB	2025-02-14 03:49
Miniconda3-py38_23.1.0-Linux-x86_64.sh	65.4 MiB	2025-02-14 03:49
Miniconda3-py38_23.1.0-MacOSX-arm64.pkg	42.6 MiB	2025-02-14 03:49
Miniconda3-py38_23.1.0-MacOSX-arm64.sh	42.9 MiB	2025-02-14 03:49

图 6.10 Miniconda 下载包

双击安装包启动安装程序,点击“Next”进入下一步,如图 6.11 所示:



图 6.11 安装 Miniconda 步骤 1

阅读许可协议后，勾选“I Agree”确认接受协议条款，如图 6.12 所示：

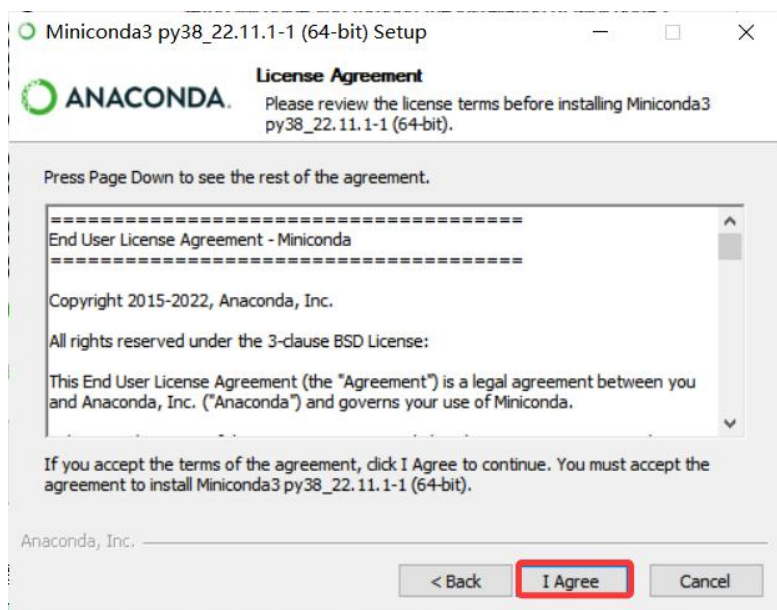


图 6.12 安装 Miniconda 步骤 2

选择安装类型，推荐选择 “Just Me (recommended)”，仅为当前用户安装，如图 6.13 所示：

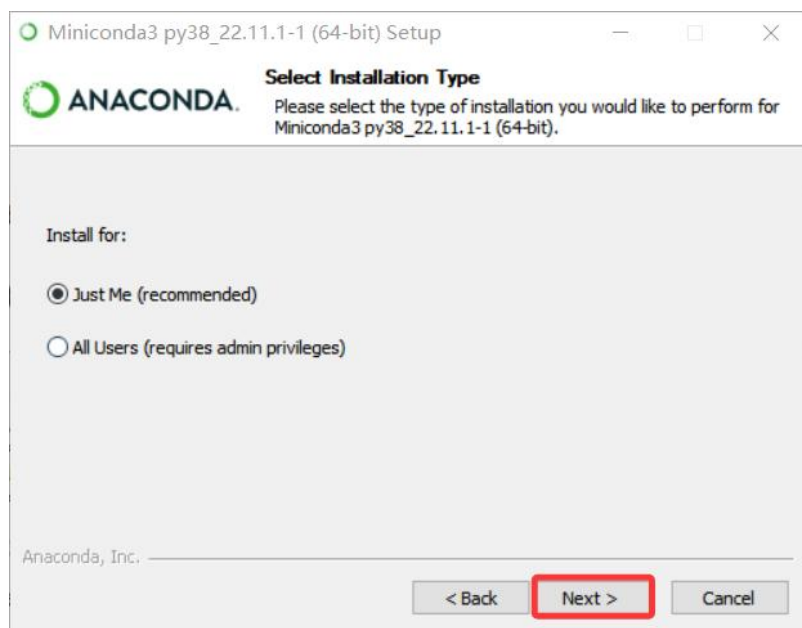


图 6.13 安装 Miniconda 步骤 3

选择安装路径，建议自定义路径（如 D:\miniconda\），确保路径无中文且剩余空间充足，如图 6.14 所示：

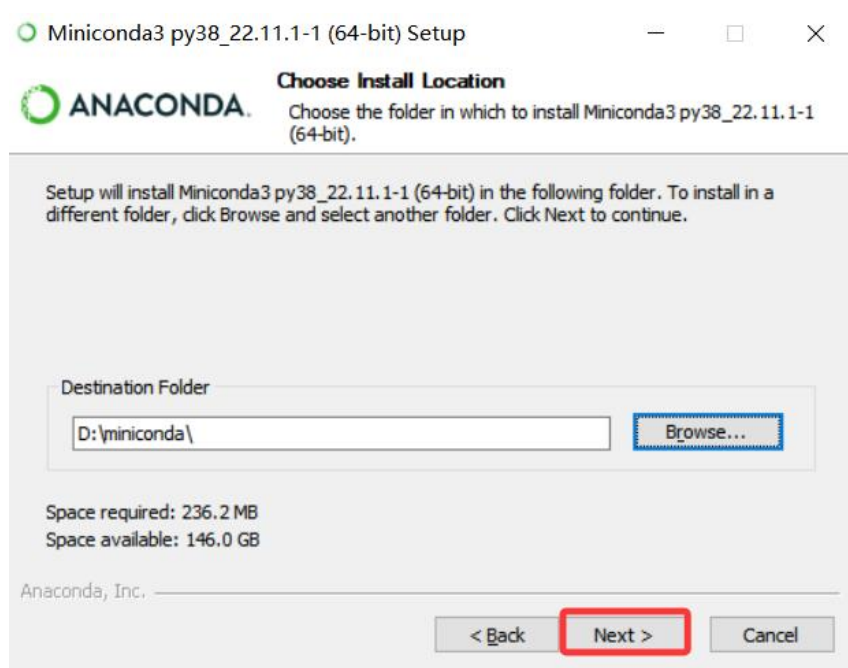


图 6.14 安装 Miniconda 步骤 4

高级安装选项保持默认设置，无需勾选“Add Miniconda3 to my PATH environment variable”，避免与系统环境冲突，如图 6.15 所示：

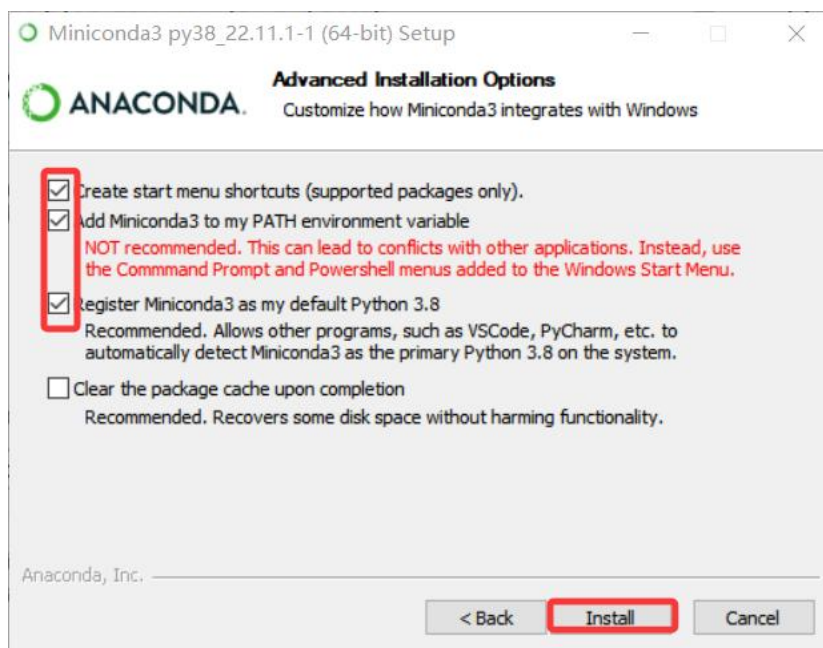


图 6.15 安装 Miniconda 步骤 5

点击 “Install” 开始安装，安装完成后点击 “Finish” 结束流程，如图 6.16 所示：



图 6.16 安装 Miniconda 步骤 6

上述操作完成后，Miniconda 安装完成。

Anaconda 安装完成后，需通过终端执行 `conda --version` 命令验证当前版本。

若显示的版本号较低（如 conda 5.5.4），可能会与 Python 3.8 环境存在兼容性问题。此情况通常源于系统预装的旧版 Anaconda，建议通过完整卸载后重新安装最新版本。版本更新后再次执行 `conda --version` 应可显示较新版本号（如 conda 22.11.1）。若现有 Anaconda 版本已支持 Python 3.8 环境配置，则无需重新安装。

步骤二：Miniconda 环境搭建

在搜索栏中输入 `cmd`，可以看到 Anaconda Prompt，点击打开，如图 6.17 所示：

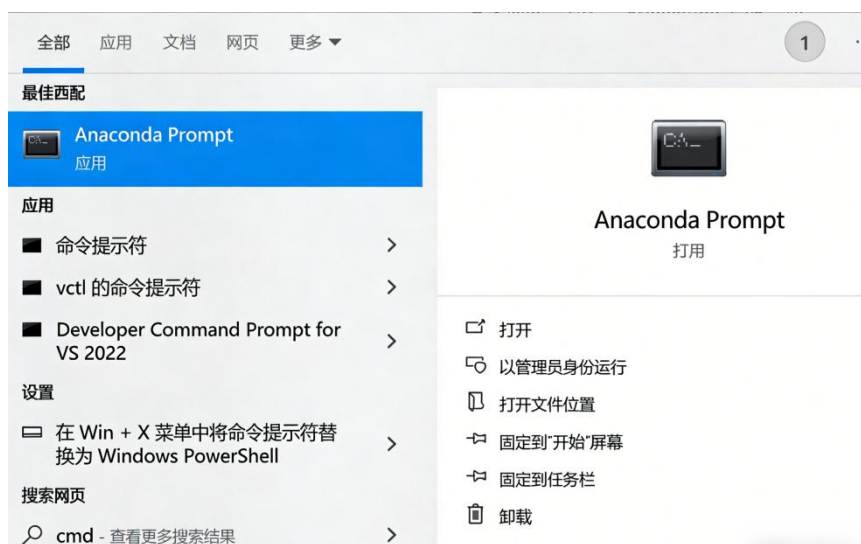


图 6.17 打开 Anaconda Prompt

输入 `conda create -n yolov5 python=3.8 -y`，如图 6.18 所示：

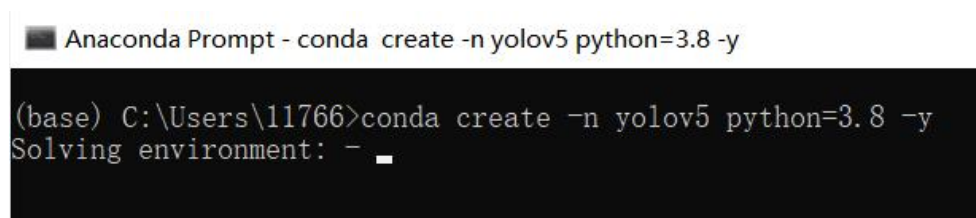


图 6.18 创建 yolov5 python=3.8

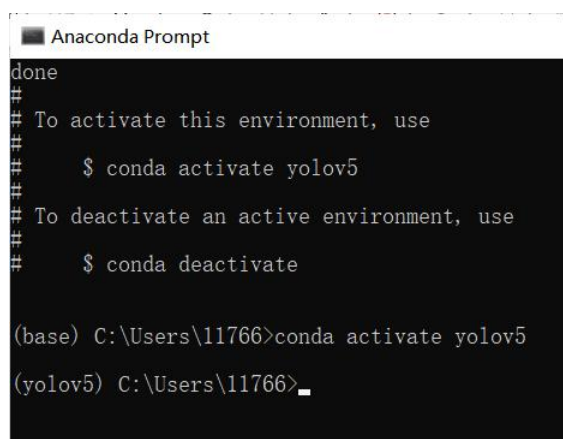
如图 6.19 所示。当出现下图的提示，则安装成功。



```
Anaconda Prompt
done
#
# To activate this environment, use
#
#     $ conda activate yolov5
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) C:\Users\11766>
```

图 6.19 创建 yolov5 python=3.8 成功

输入 `conda activate yolov5` 来激活环境，如图 6.20 所示：



```
Anaconda Prompt
done
#
# To activate this environment, use
#
#     $ conda activate yolov5
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) C:\Users\11766>conda activate yolov5
(yolov5) C:\Users\11766>_
```

图 6.20 激活 yolov5 环境

步骤三：配置 PyPI 镜像源

下面要配置 PyPI 镜像源，国内直接连接 PyPI 官方源速度慢，易超时失败，YOLOv5 训练需安装大量依赖（如 Torch、OpenCV 等），配置镜像源可节省 90% 以上安装时间。

进入清华镜像源（<https://mirrors.tuna.tsinghua.edu.cn/help/pypi/>），如图 6.21 所示：

PyPI 软件仓库

线路选择 自动
☐ 是否使用 sudo

服务域名

如果您正在使用 TUNA 镜像站，可以使用 `pipi.tuna.tsinghua.edu.cn` 替换下面所有的 `mirrors.tuna.tsinghua.edu.cn/pypi/web`。

BFSU 镜像站没有为 PyPI 提供独立服务域名。

PyPI 镜像在每次同步成功后间隔 5 分钟同步一次。

pip

临时使用

```
pip install -i https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple some-package
```

注意，`simple` 不能少。

pip 要求使用 `https`，因此需要 `https` 而不是 `http`

设为默认

升级 pip 到最新的版本后进行配置：

```
python -m pip install --upgrade pip
pip config set global.index-url https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
```

如果您到 pip 默认源的网络连接较差，临时使用本镜像站来升级 pip：

```
python -m pip install -i https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple --upgrade pip
```

配置多个镜像源

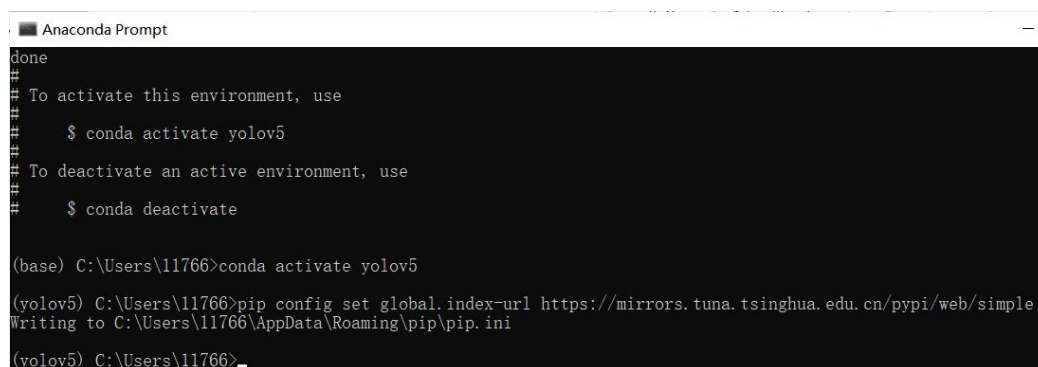
如果您想配置多个镜像源平衡负载，可在已经替换 `index-url` 的情况下通过以下方式继续增加源站：

图 6.21 清华源 PyPI 软件仓库

在终端中继续输入上图中的命令，将 PyPI（Python Package Index）的默认源替换为清华大学镜像源。

pip config set global.index-url <https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple>

出现 Writing to C:\Users\11766\AppData\Roaming\pip\pip.ini, 则说明替换成功，如图 6.22 所示：



```

Anaconda Prompt
done
# To activate this environment, use
#
# $ conda activate yolov5
#
# To deactivate an active environment, use
#
# $ conda deactivate
#

(base) C:\Users\11766>conda activate yolov5
(yolov5) C:\Users\11766>pip config set global.index-url https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
Writing to C:\Users\11766\AppData\Roaming\pip\pip.ini
(yolov5) C:\Users\11766>

```

图 6.22 配置清华园镜像

步骤四：PyTorch 安装

在安装 PyTorch 前，需通过 NVIDIA 控制面板确认系统 CUDA 版本以确保

兼容性，如图 6.23 所示。具体而言，可在桌面右键打开 NVIDIA 控制面板。



图 6.23 打开 NVIDIA 控制面板

点击查看系统信息，如图 6.24 所示：



图 6.24 查看系统信息

通过查看系统信息中的组件选项卡获取当前 CUDA 版本号（如 12.6.65），

该版本代表驱动程序支持的最高 CUDA 版本，如图 6.25 所示。基于此，安装 PyTorch 时应选择 CUDA 版本号不超过该数值的发行版，以保证框架的正常运行。

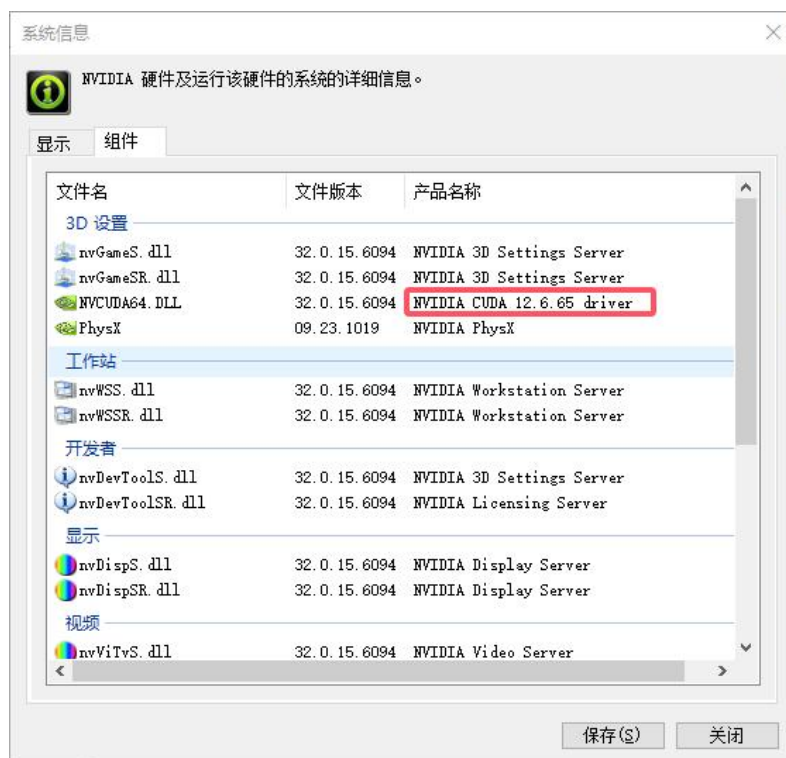


图 6.25 查看 CUDA 版本

如图 6.26 所示。进入 PyTorch 官网，选择之前的版本。

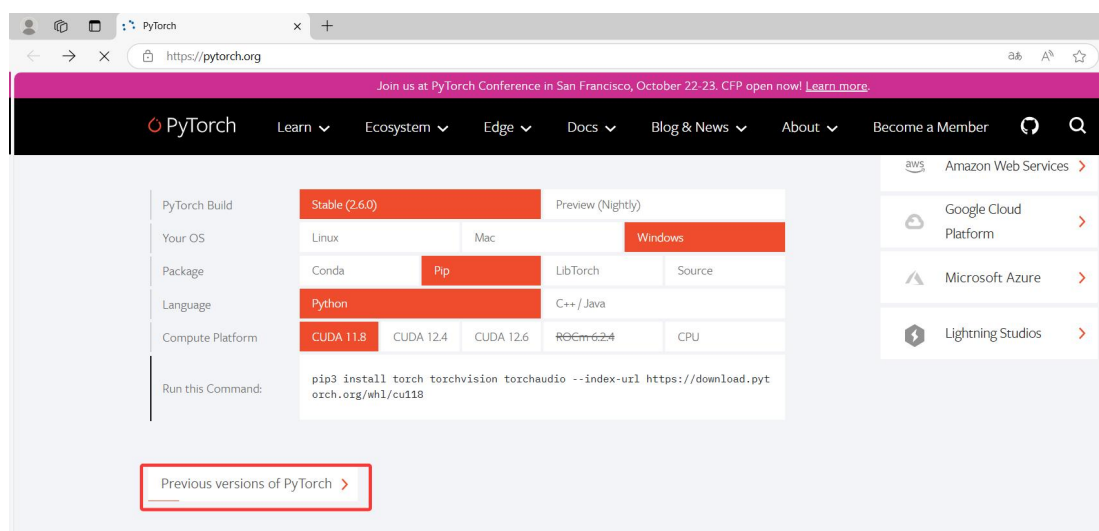


图 6.26 进入 PyTorch 官网

查看官网依赖版本，如图 6.27 所示：

Linux and Windows

```
# CUDA 10.2
pip3 install torch==1.8.2 torchvision==0.9.2 torchaudio==0.8.2 --extra-index-url https://download.pytorch.org/whl/lts/1.8/cu102

# CUDA 11.1
pip3 install torch==1.8.2 torchvision==0.9.2 torchaudio==0.8.2 --extra-index-url https://download.pytorch.org/whl/lts/1.8/cu111

# CPU Only
pip3 install torch==1.8.2 torchvision==0.9.2 torchaudio==0.8.2 --extra-index-url https://download.pytorch.org/whl/lts/1.8/cpu

# ROCm5.x

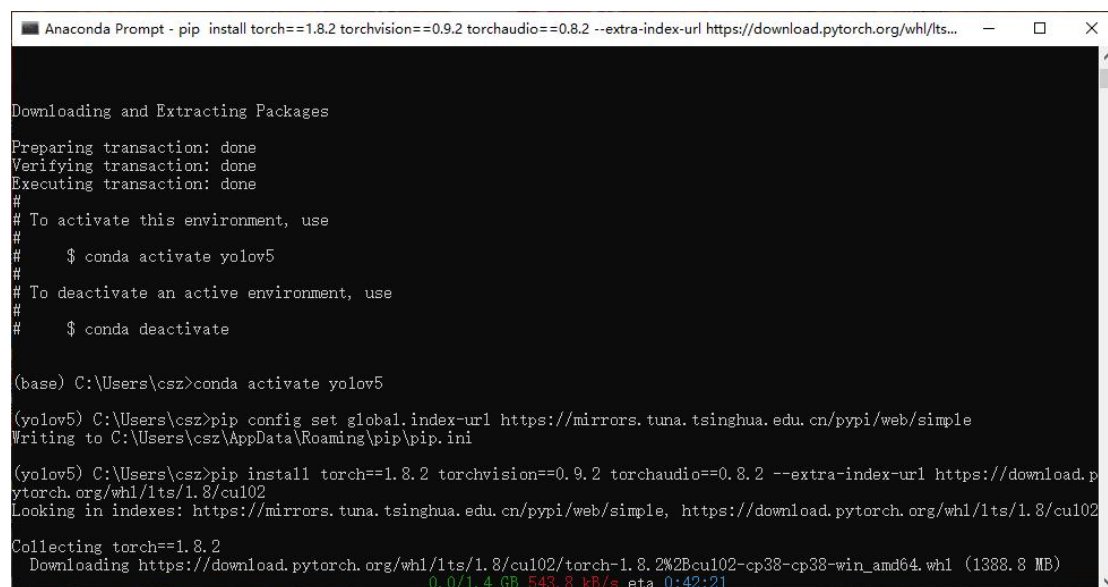
Not supported in LTS.
```

v1.8.1

图 6.27 查看官网依赖版本

(`pip install torch==1.8.2 torchvision==0.9.2 torchaudio==0.8.2 --extra-index-url https://download.pytorch.org/whl/lts/1.8/cu102`)

安装 PyTorch，如图 6.28 所示：



```
Anaconda Prompt - pip install torch==1.8.2 torchvision==0.9.2 torchaudio==0.8.2 --extra-index-url https://download.pytorch.org/whl/lts/1.8/cu102

Downloading and Extracting Packages
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#   $ conda activate yolov5
#
# To deactivate an active environment, use
#
#   $ conda deactivate

(base) C:\Users\csz>conda activate yolov5

(yolov5) C:\Users\csz>pip config set global.index-url https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
Writing to C:\Users\csz\AppData\Roaming\pip\pip.ini

(yolov5) C:\Users\csz>pip install torch==1.8.2 torchvision==0.9.2 torchaudio==0.8.2 --extra-index-url https://download.pytorch.org/whl/lts/1.8/cu102
Looking in indexes: https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple, https://download.pytorch.org/whl/lts/1.8/cu102
Collecting torch==1.8.2
  Downloading https://download.pytorch.org/whl/lts/1.8/cu102/torch-1.8.2%2Bcu102-cp38-cp38-win_amd64.whl (1338.8 MB)
    0.0/1.4 GB 543.8 kB/s eta 0:42:21
```

图 6.28 安装 PyTorch

这个过程可能需要一段时间，直至终端显示“Successfully installed”即表示安装完成。

步骤五：安装 YOLOv5 依赖

将实验包中的 yolov5 文件夹复制到桌面，如图 6.29 所示：

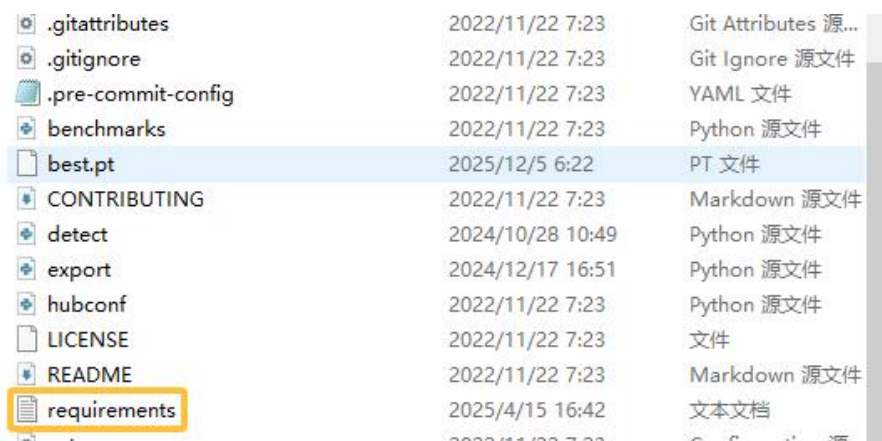


图 6.29 找到 requirements

在终端中，进入桌面，进入 yolov5-7.0。

在终端输入 `pip install -r requirements.txt`，安装文本文件中的依赖，如图 6.30 所示：

```

Anaconda Prompt
. 2) (8.3.0)
(yolov5) C:\Users\csz>cd Desktop
(yolov5) C:\Users\csz\Desktop>cd yolov5-7.0
(yolov5) C:\Users\csz\Desktop\yolov5-7.0>pip install -r requirements.txt
WARNING: Ignored distutils configs in setup.cfg due to encoding errors.
Looking in indexes: https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
Requirement already satisfied: gitpython>=3.1.30 in c:\users\csz\.conda\envs\yolov5\lib\site-packages (from -r requirements.txt (line 5)) (3.1.43)
Requirement already satisfied: ipython in c:\users\csz\.conda\envs\yolov5\lib\site-packages (from -r requirements.txt (line 6)) (8.12.3)
Requirement already satisfied: matplotlib>=3.2.2 in c:\users\csz\.conda\envs\yolov5\lib\site-packages (from -r requirements.txt (line 7)) (3.7.5)
Requirement already satisfied: numpy==1.20.3 in c:\users\csz\.conda\envs\yolov5\lib\site-packages (from -r requirements.txt (line 8)) (1.20.3)
Requirement already satisfied: opencv-python>=4.1.1 in c:\users\csz\.conda\envs\yolov5\lib\site-packages (from -r requirements.txt (line 9)) (4.10.0.84)
Requirement already satisfied: Pillow>=7.1.2 in c:\users\csz\.conda\envs\yolov5\lib\site-packages (from -r requirements.txt (line 10)) (8.3.0)
Requirement already satisfied: psutil in c:\users\csz\.conda\envs\yolov5\lib\site-packages (from -r requirements.txt (line 11)) (6.1.0)
Requirement already satisfied: PyYAML>=5.3.1 in c:\users\csz\.conda\envs\yolov5\lib\site-packages (from -r requirements.txt (line 12)) (6.0.2)
Requirement already satisfied: requests>=2.23.0 in c:\users\csz\.conda\envs\yolov5\lib\site-packages (from -r requirements.txt (line 13)) (2.32.3)
Requirement already satisfied: scipy>=1.4.1 in c:\users\csz\.conda\envs\yolov5\lib\site-packages (from -r requirements.txt (line 14)) (1.10.1)
Requirement already satisfied: thop>=0.1.1 in c:\users\csz\.conda\envs\yolov5\lib\site-packages (from -r requirements.txt (line 15)) (0.1.1)

```

图 6.30 安装相关依赖

因为案例中的电脑之前已经安装完成，所有并没有安装进度。

在同学们的实践中，如果出现下图的提示，则表示安装完成，如图 6.31 所示：

图 6.31 相关依赖安装完成

测试 detect.py, 如图 6.32 所示:

图 6.32 测试 detect.py

关于安装中的其他问题

1.pip 和 conda 到使用哪个

在一个单独的环境中, 能使用 pip 就尽量使用 pip, 实在有问题的情况, 再使用 conda 进行安装, 不要来回混淆。

2.CUDA 是否必须安装

如果只需要训练、简单推理, 则无需单独安装 CUDA, 直接在官网搜索对应的版本, 通过 pip 安装即可。

如果需要编译、部署等，那么还是需要单独安装 CUDA 的，这部分大家可以自行百度。

3. 显卡 16XX 的安装问题

如果想一切顺利运行，尽量安装 cu10.x 的 pytorch 版本，否则会出现预测失效、训练过程中 Loss 为 nan、各项指标均为 0 的情况。

4. 如果必须安装 cu11.x 的版本，则需要修改代码中半精度的部分，才能避免上述问题。

5. 显卡 30XX 的安装问题

30 系列的显卡不再支持 cuda10，因此务必安装 cu11.x 的 pytorch 版本。

步骤六：配置 Visual Studio Code 开发环境

安装 Visual Studio Code，安装和配置教程可网上自行查找，安装 Python 扩展。下载地址：（<https://code.visualstudio.com/>）

安装完成后，将我们给出的 yolov5 文件用 Visual Studio Code 打开，

点击 VS Code 右下角的“选择解释器”，或通过“查看”→“命令面板”→输入“Python: 选择解释器”，选择“yolov5”虚拟环境对应的 Python 解释器（路径通常为“C:\Users\用户名\.conda\envs\yolov5\python.exe”），如图 6.33 所示：

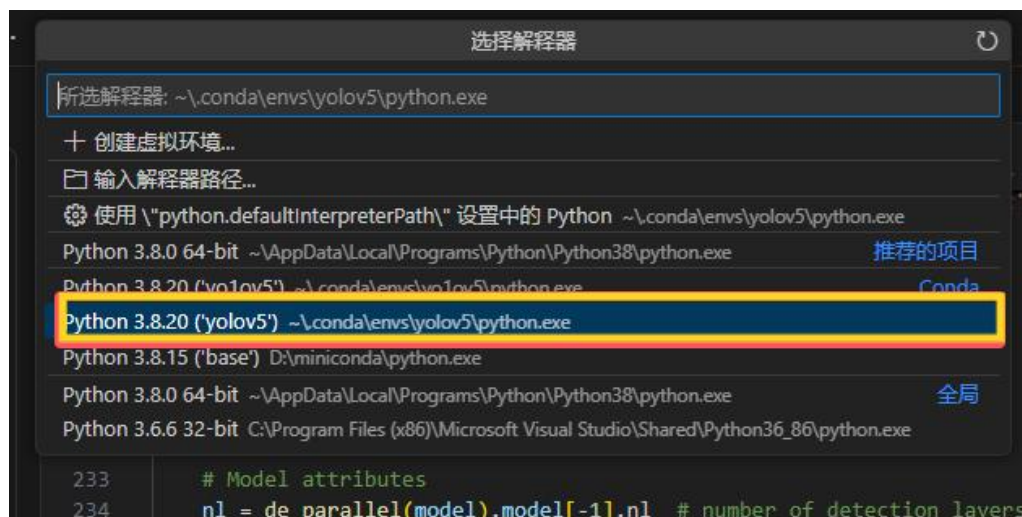


图 6.33 VS code 选择 Python 环境

步骤七：配置数据集 YAML 文件

如图 6.34 所示。修改数据集配置文件，编辑 YAML 文件，在已经给出的 data 文件中找到 coco128.yaml，复制一份重新命名，案例中命名为 ccsszz.yaml。

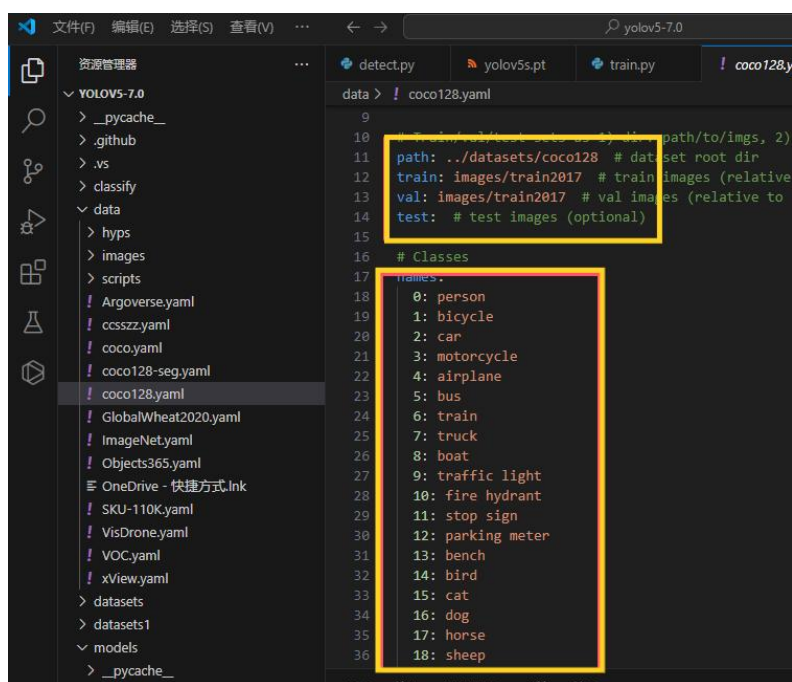


图 6.34 配置 YAML 文件

按照我们自己的训练集和验证集路径以及类别数和类别名称来更改，如图 6.35 所示：

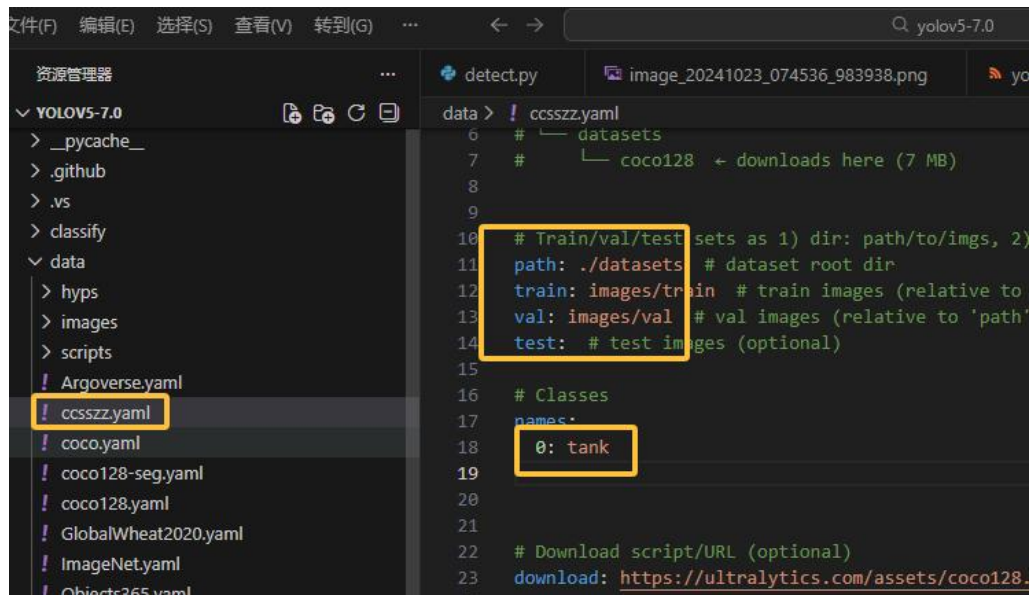


图 6.35 YAML 配置后

步骤八：熟悉训练参数与启动训练

打开 train.py 文件，查看 parse_opt 函数中的关键训练参数，如图 6.36 所示：

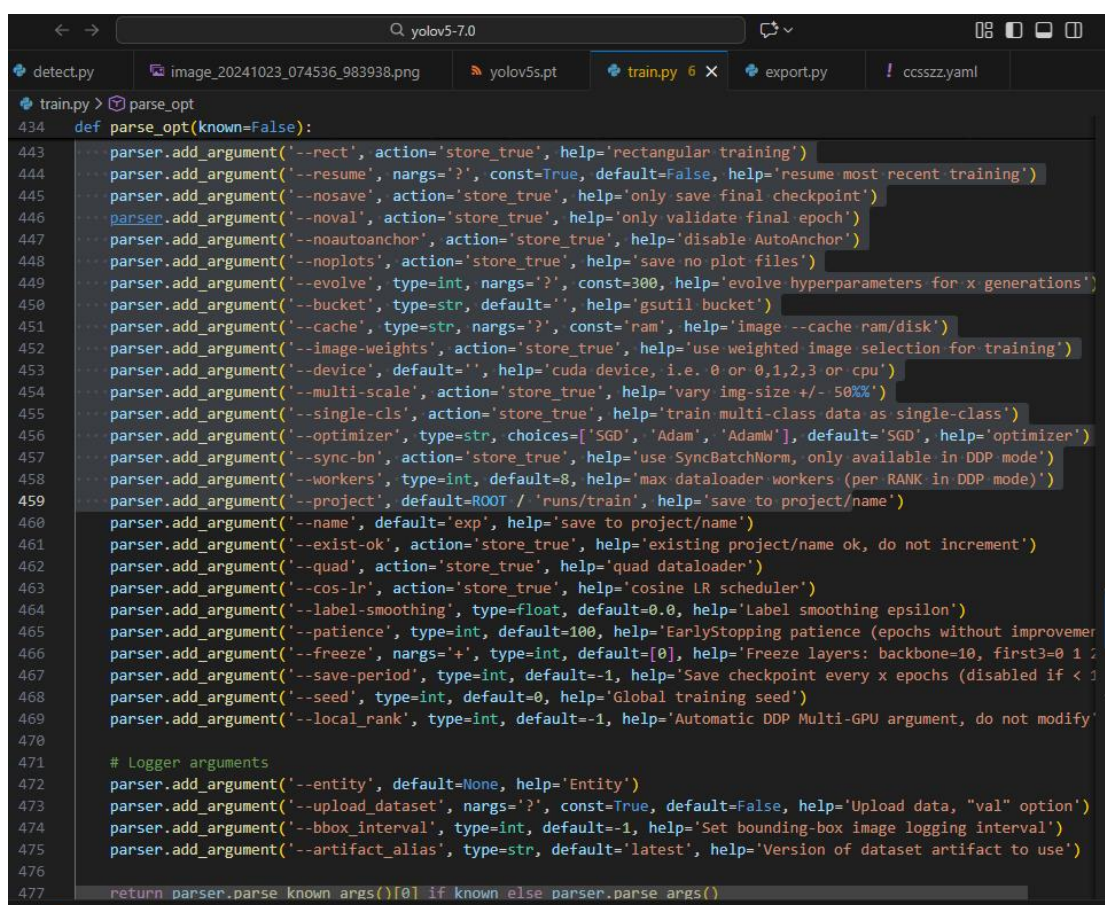


图 6.36 训练脚本中关键参数

训练脚本中关键参数如表 6.1 所示：

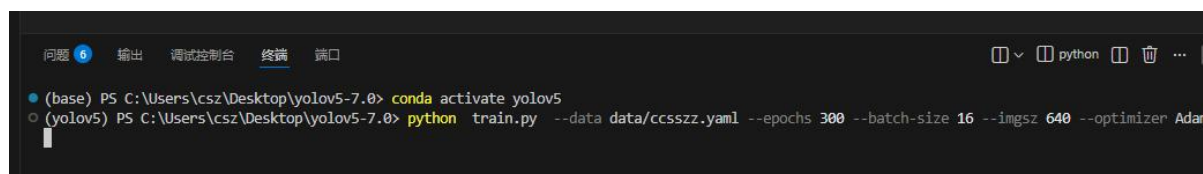
表 6.1 训练脚本中关键参数

参数	示例值	说明
--data	data/ccsszz.yaml	数据集配置文件路径，需指定训练/验证集路径、类别数和名称。
--weights	yolov5s.pt	预训练模型权重，若为空（"）则从零开始训练。推荐使用预训练权重加速收敛。
--epochs	300	训练轮次，坦克检测通常需要 200-300 轮（数据集较小时可减少）。

--batch-size	32	批大小，根据 GPU 显存调整（显存不足时降低此值）。
--imgsz	640	输入图像尺寸，必须是 32 的倍数。较大尺寸（如 640）可提升检测精度但增加计算量。
--optimizer	Adam	优化器选择： - SGD（默认）：适合大数据集，需调参 - Adam：收敛快，适合小数据集 - AdamW：改进版 Adam，防过拟合
--cos-lr	True	余弦学习率调度，使学习率从初始值平滑下降至最终值（--lrf），推荐启用。
--cos-lr	True	余弦学习率调度，使学习率从初始值平滑下降至最终值（--lrf），推荐启用。
--label-smoothing	0.1	标签平滑，减轻过拟合（范围 0-0.2）。坦克检测建议 0.05-0.1。
--multi-scale	True	多尺度训练，随机调整输入图像尺寸（±50%），提升模型泛化能力。
--mosaic	1.0	马赛克增强（默认启用），将 4 张图像拼接训练，提升小目标检测能力。
-hsv-h	0.015	色调增强强度（范围 0-0.1），模拟不同光照条件。

--cfg	yolov5s.yaml	模型配置文件，定义网络结构。可选 yolov5s/m/l/x（小/中/大/超大模型）。
--freeze	[0]	冻结层，冻结前 N 层（如 --freeze 10 冻结骨干网络），减少训练参数。
--resume	True	恢复训练，从上次中断的检查点（last.pt）继续训练。
--patience	100	早停机制，若验证集精度在 N 轮内未提升则终止训练（防过拟合）。
--device	0	训练设备，0 表示 GPU 0 号卡，cpu 表示 CPU 训练。
--project	runs/train	实验保存目录，所有输出（权重、日志、图表）将存储在此路径下。
--name	exp	实验名称，自动生成子目录（如 exp2, exp3）

终端模型训练命令如图 6.37 所示：



```

(base) PS C:\Users\csz\Desktop\yolov5-7.0> conda activate yolov5
(yolov5) PS C:\Users\csz\Desktop\yolov5-7.0> python train.py --data data/ccsszz.yaml --epochs 300 --batch-size 16 --imgsz 640 --optimizer Adam

```

图 6.37 终端模型训练命令

步骤九：模型训练

坦克检测推荐配置：python train.py --data data/ccsszz.yaml --weights yolov5s.pt --epochs 300 --batch-size 32 --imgsz 64 --optimizer Adam --cos-lr --label-smoothing 0.1

案例中配置：python train.py --data data/ccsszz.yaml --epochs 300 --batch-size 32 --imgsz 640 --optimizer Adam

步骤十：注意事项

1.如果显存不足可以适当降低--batch-size（如 16）或--imgsz（如 416）。

2.如果出现过拟合可以增加--label-smoothing (0.1→0.2) 或减少--epochs。

3.所有未在命令行中显式指定的参数都会使用 parse_opt()函数中定义的默认值, 通过终端直接指定的参数(如--epochs 300)优先级最高, 会覆盖其他所有来源的参数值, 在 train.py 的 parse_opt()函数中定义的默认值是第二优先级。

在图中我们可以看到实时训练指标, 当出现 Starting training for 300 epochs...表明训练流程已正常启动, 如图 6.38 所示:

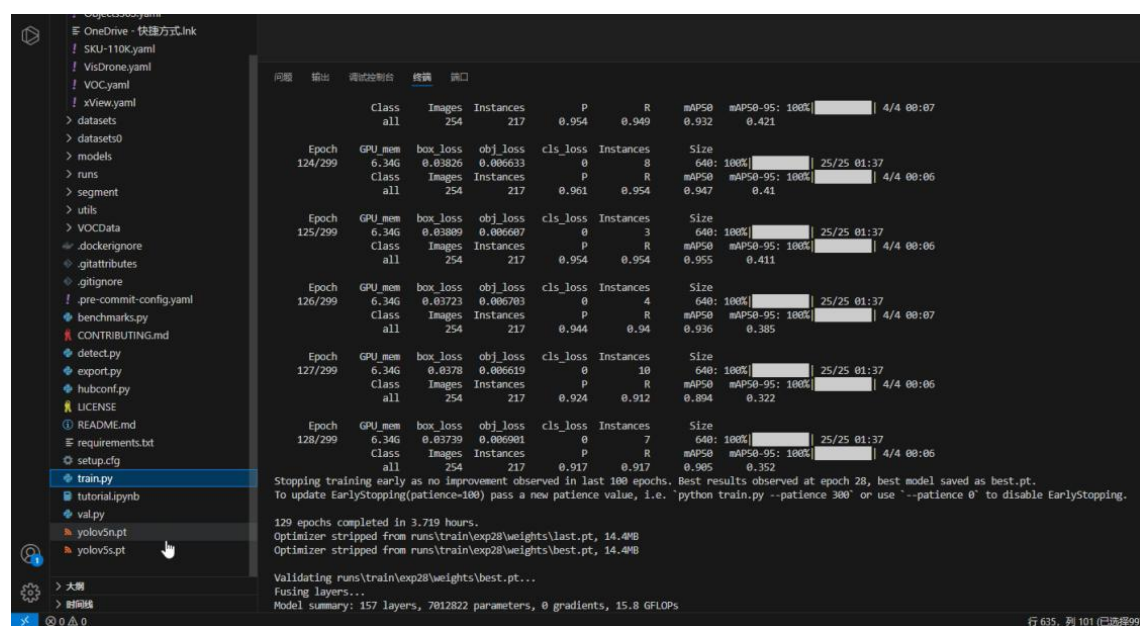
```
AMP: checks passed
optimizer: Adam(lr=0.01) with parameter groups 57 weight(decay=0.0), 60 weight(decay=0.0005), 60 bias
train: Scanning C:\Users\csz\Desktop\yolov5-7.0\datasets\labels\train... 698 images, 85 backgrounds, 0 corrupt: 100%
train: WARNING Cache directory C:\Users\csz\Desktop\yolov5-7.0\datasets\labels is not writeable: [winError 183] : 'C:\Users\csz\Desktop\yolov5-7.0\datasets\labels\train.cache'
val: Scanning C:\Users\csz\Desktop\yolov5-7.0\datasets\labels\val... 253 images, 37 backgrounds, 0 corrupt: 100%
val: WARNING Cache directory C:\Users\csz\Desktop\yolov5-7.0\datasets\labels is not writeable: [winError 183] : 'C:\Users\csz\Desktop\yolov5-7.0\datasets\labels\val.cache'

AutoAnchor: 5.71 anchors/target, 1.000 Best Possible Recall (BPR). Current anchors are a good fit to dataset
Plotting labels to runs\train\exp32\labels.jpg...
Image sizes 640 train, 640 val
Using 6 dataloader workers
Logging results to runs\train\exp32
Starting training for 300 epochs...
```

epoch	GPU_mem	box_loss	obj_loss	cls_loss	Instances	Size
0/299	3.5G	0.09223	0.02029	0	17	640: 32% 14/44 00:54

图 6.38 模型训练轮数进行

等待若干小时后, 模型就训练完成了, 如图 6.39 所示:



```
Class Images Instances P R mAP50 mAP50-95: 100% 4/4 00:07
all 254 217 0.954 0.949 0.932 0.421

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
124/299 6.34G 0.03826 0.006633 0 8 640: 100% 25/25 01:37
Class Images Instances P R mAP50 mAP50-95: 100% 4/4 00:06
all 254 217 0.961 0.954 0.947 0.41

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
125/299 6.34G 0.03089 0.006687 0 3 640: 100% 25/25 01:37
Class Images Instances P R mAP50 mAP50-95: 100% 4/4 00:06
all 254 217 0.954 0.954 0.955 0.411

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
126/299 6.34G 0.03723 0.006783 0 4 640: 100% 25/25 01:37
Class Images Instances P R mAP50 mAP50-95: 100% 4/4 00:07
all 254 217 0.944 0.94 0.936 0.385

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
127/299 6.34G 0.0378 0.006619 0 10 640: 100% 25/25 01:37
Class Images Instances P R mAP50 mAP50-95: 100% 4/4 00:06
all 254 217 0.924 0.912 0.894 0.322

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
128/299 6.34G 0.03739 0.006981 0 7 640: 100% 25/25 01:37
Class Images Instances P R mAP50 mAP50-95: 100% 4/4 00:06
all 254 217 0.917 0.917 0.905 0.352

Stopping training early as no improvement observed in last 100 epochs. Best results observed at epoch 28, best model saved as best.pt.
To update EarlyStopping(patience=100) pass a new patience value, i.e. 'python train.py --patience 300' or use '--patience 0' to disable EarlyStopping.
129 epochs completed in 3.719 hours.
Optimizer stripped from runs\train\exp28\weights\last.pt, 14.4MB
Optimizer stripped from runs\train\exp28\weights\best.pt, 14.4MB
Validating runs\train\exp28\weights\best.pt...
Fusing layers...
Model summary: 157 layers, 7812822 parameters, 0 gradients, 15.8 GFLOPs
```

图 6.39 模型训练完成

训练终止后，系统自动生成以下文件，存储于 runs/train/exp24（exp 是你最近的训练结果，案例中是 exp28）

exp28/

- |— weights/
 - | |— best.pt # 验证集 mAP 最高的模型权重
 - | |— last.pt # 最终 epoch 的完整模型（含优化器状态）
 - | |— epoch23.pt # 早停时记录的最佳 epoch 权重
- |— results.png # 损失和 mAP 变化曲线
- |— labels.jpg # 训练标签分布可视化
- |— args.yaml # 训练参数备份

其中 best.pt 是要进行模型转化的文件。